Toxic Comment Classification:
A Machine Learning Approach

This report presents an end-to-end machine learning pipeline for detecting toxic comments online, focusing on feature engineering, model interpretability, and promoting healthier digital conversations.

https://github.com/Morris-CP/Toxic-comments-categorized/tree/main

Table of Contents

Introduction

1.1 Project Overview

This project is based on the Kaggle Toxic Comment Classification Challenge and aims to build a multi-label text classification model to automatically identify various types of harmful content in user comments, including toxic, severe toxic, obscene, threat, insult, and identity-based hate speech.

The project uses real comments from Wikipedia Talk Pages, addressing practical challenges commonly found in user-generated text data, such as high textual noise, imbalanced labels, and samples that may have multiple labels simultaneously. The model outputs the probability of each type of toxicity, and average ROC AUC is used as the evaluation metric to comprehensively measure the model's ability to detect different types of toxic behavior.

This project simulates the real-world needs of content moderation on platforms and serves as a comprehensive practice case for text modeling, multi-label classification, and model evaluation.

1.2 Dataset Description

The dataset comes from the historical comments on Wikipedia Talk Pages, containing a large amount of real user-generated content. Each sample consists of a piece of English text and its corresponding toxicity labels.
The labels are multi-label, covering six types of toxicity:

- toxic
- severe_toxic
- obscene
- threat
- insult
- identity_hate

The label distribution is highly imbalanced, with some high-risk categories having very few samples, posing challenges to model stability and generalization.Additionally, the text contains colloquial expressions, spelling errors, and offensive words, reflecting the complexity of real-world online comment data.

- train.csv – Training set with binary annotations. (159571, 8)
- test.csv – Test set; you need to predict the toxicity probabilities for these comments. Some comments are excluded from scoring to prevent manual labeling. (153164, 2)

- test_labels.csv – Labels for the test set; a value of -1 indicates that the label was not used for scoring (note: this file was added after the competition ended). (63978, 7)

Analysis Approach and Methodology

2.1 Overall Analysis Workflow

1. **Problem Analysis**

   The task is defined as a multi-label text classification problem, where the model predicts the probability of each type of toxicity for every comment, rather than outputting a single class label.

2. **Import Libraries and Load Data**

   Import the necessary Python libraries for data analysis and machine learning, then load the training and test datasets for basic data reading.

3. **Descriptive Statistics**

   Perform descriptive statistical analysis to understand the dataset size, text length distribution, and basic characteristics of each toxicity label.

4. **Exploratory Data Analysis (EDA)**

   Explore data features using various visualization techniques, including:

- Distribution of each toxicity label in the training set
- Proportion of comments containing multiple toxicity labels simultaneously
- Co-occurrence relationships among different toxicity labels
- High-contributing keywords for each toxicity label (word clouds)
- Main tools used include describe(), barplot, heatmap, and wordcloud.

5. **Feature Engineering**

   Transform raw text into numerical features using text vectorization methods:

- CountVectorizer: Constructs basic word frequency features
- TfidfVectorizer: Incorporates inverse document frequency to enhance feature discriminability

6. **Text Preprocessing**

   Clean and standardize raw text data, including:

- Regex-based cleaning (re)
- Removing irrelevant symbols (sub)
- Tokenization
- Lemmatization (WordNetLemmatizer)

7. **Text Feature Extraction**

   Convert cleaned text into sparse vector representations to serve as input features for subsequent model training.

8. **Modeling**

Baseline Models

Construct multiple baseline models for comparison:

- Multinomial Naive Bayes
- Logistic Regression
- Linear Support Vector Classifier (Linear SVC)

Evaluate model performance using 10-fold cross-validation and select the best-performing model based on average ROC AUC.

Enhanced Models

On top of baseline models, introduce:

- AdaBoost
- GBDT
- XGBoost

Select the final model based on AUC performance on the validation set.

9. **Ensemble Learning**

   Use VotingClassifier to combine multiple complementary models, improving overall prediction performance and model stability.

10. **Prediction**

    Use the final model to generate predictions on the test set, output probabilities for each toxicity label, and produce a submission file that meets competition requirements.

2.2 Methodology and Model Selection Rationale

This project models the task as a **multi-label text classification problem**, using a **One-vs-Rest** approach to predict the probability of each toxicity label independently. The evaluation metric is **average ROC AUC**, suitable for imbalanced labels and emphasizing ranking ability.

For feature representation, **CountVectorizer** and **TF-IDF Vectorizer** are used to construct text features, balancing computational efficiency and model stability, and serving as unified input for multiple models.

Model selection follows a **simple-to-complex principle**:

First, construct baseline models using **Multinomial Naive Bayes**, **Logistic Regression**, and **Linear SVC**, establishing performance baselines through cross-validation.

Then, enhance model expressiveness with **AdaBoost**, **GBDT**, and **XGBoost**, selecting models based on **AUC**.

Finally, combine models using **VotingClassifier** to improve overall performance and generalization ability.

Data Analysis and Feature Engineering

3.1 Data Quality Assessment

Before modeling, a basic data quality assessment was conducted to ensure usability and consistency. The main checks included dataset size, missing values, validity of label values, and completeness of text fields.

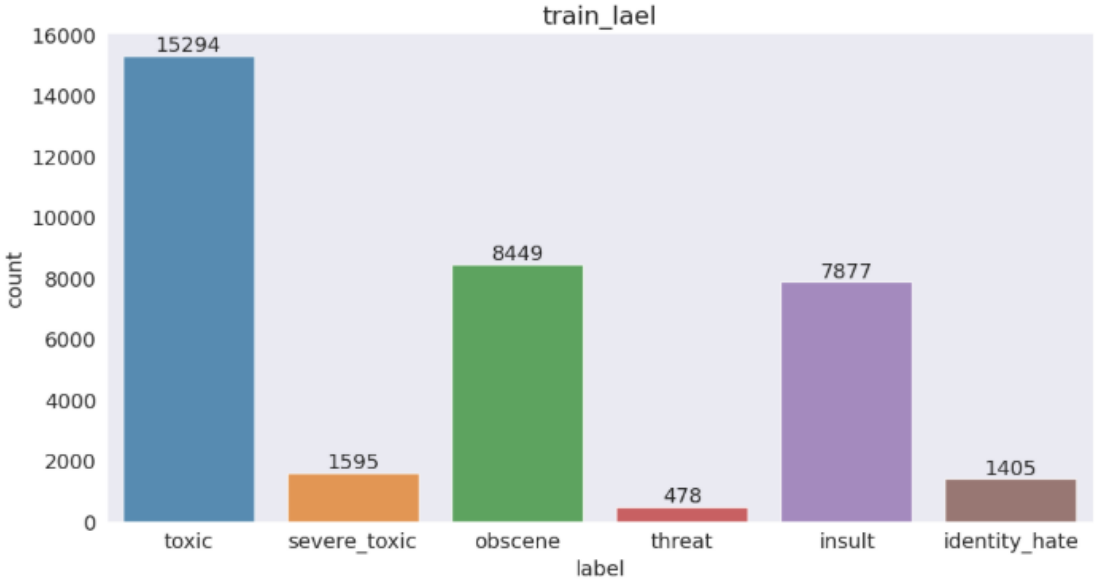
The results indicate that the comment text fields have no missing values, label fields contain only 0/1 values, and the overall data structure is clean. Therefore, the dataset is suitable for multi-label text classification modeling.

Additionally, the distribution of comment text lengths was analyzed to understand text characteristics and provide guidance for subsequent feature engineering.

3.2 Exploratory Data Analysis (EDA)

3.2.1 Label Distribution Analysis

The distribution of each toxicity label in the training set was first examined. The results show a clear imbalance among different toxicity categories, with labels such as toxic and insult having a larger number of samples, while threat and severe_toxic have comparatively fewer samples.



This phenomenon indicates that during model training, attention must be paid to the recognition of low-frequency labels to avoid a model that performs well only on high-frequency categories.

3.2.2 Multi-Label Co-Occurrence Analysis

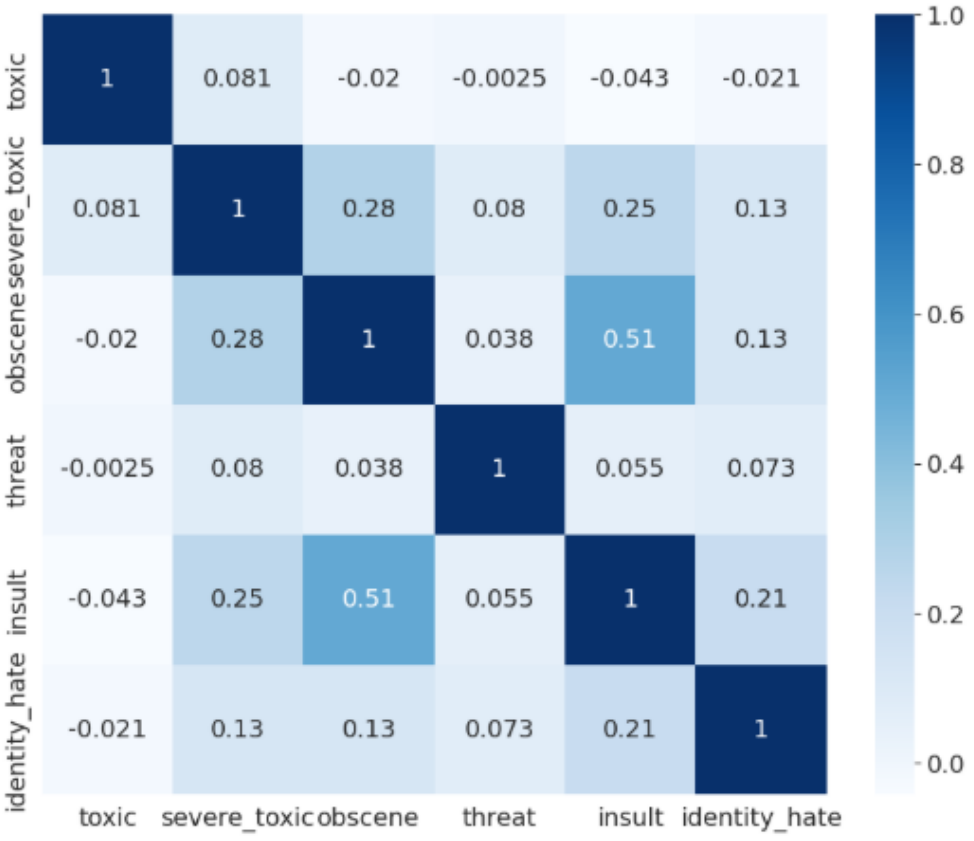
Since this task involves multi-label classification, we further analyzed cases where a single comment contains multiple toxicity labels. The results show that some comments exhibit two or

https://github.com/Morris-CP/Toxic-comments-categorized/tree/main

more toxic characteristics simultaneously, confirming the rationale for modeling this problem as a multi-label classification task.

3.2.3 Label Co-Occurrence Analysis

Compute the co-occurrence frequency between different toxicity labels and visualize the results using a heatmap to analyze the relationships between labels. The results show that *toxic* frequently co-occurs with *insult* and *obscene*, whereas *threat* shows relatively weak correlations with other labels.

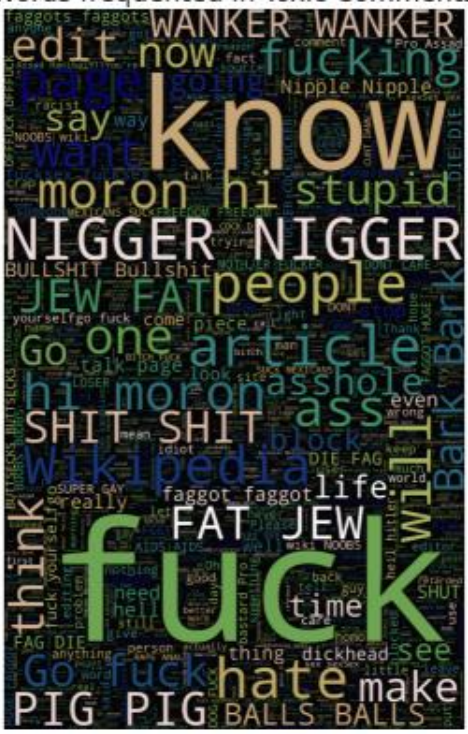


https://github.com/Morris-CP/Toxic-comments-categorized/tree/main

3.2.4 Keywords and Text Feature Analysis

For each toxicity label, high-frequency words are counted and displayed using word clouds to visually observe the typical language characteristics of each toxicity type. This analysis helps understand the discriminative cues that the model may learn.

Words frequented in Toxic Comments

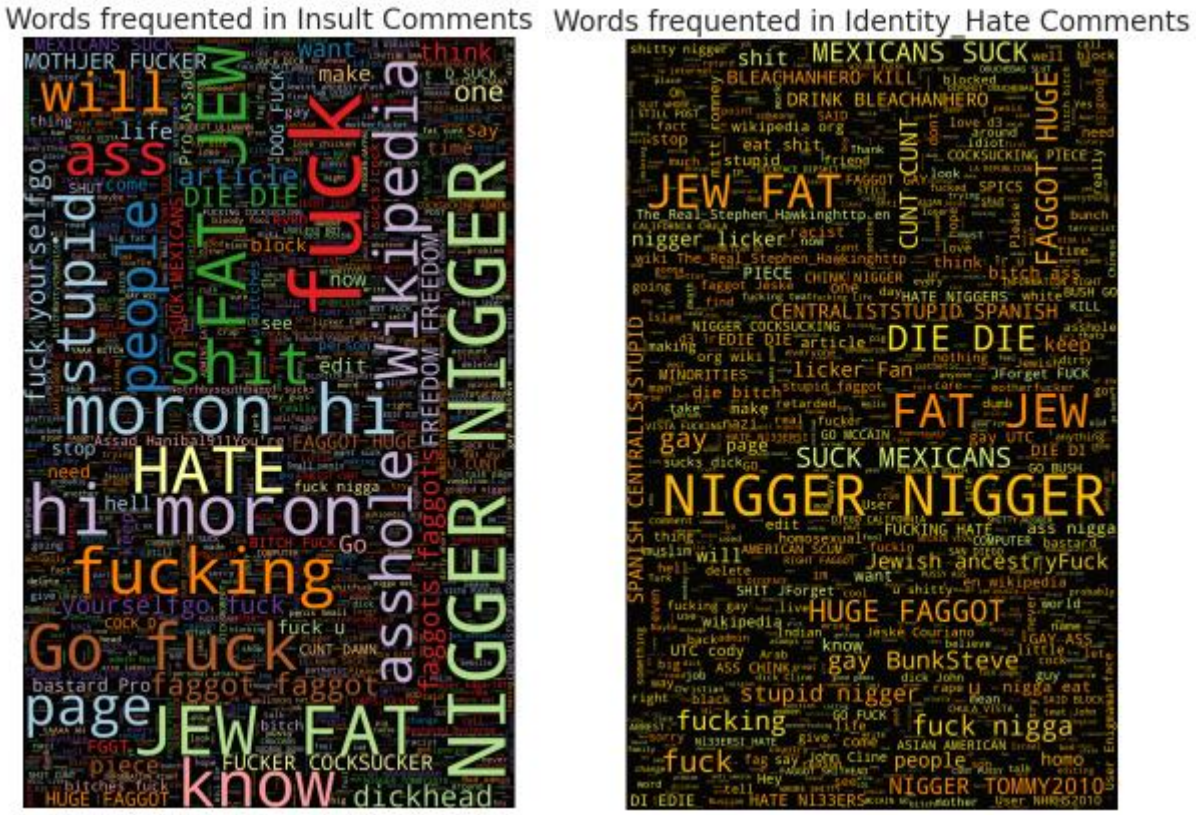


Words frequented in Severe_Toxic Comments

Words frequented in Obscene Comments

Words frequented in Threat Comments

https://github.com/Morris-CP/Toxic-comments-categorized/tree/main



3.3 Feature Engineering Strategy

Before vectorizing the text, the following preprocessing steps are performed sequentially:

1. **Lowercasing**
   Convert all text to lowercase to avoid treating the same word with different cases as distinct features.
2. **Removing punctuation and numbers**
   Use regular expressions to remove punctuation, numbers, and line breaks to reduce non-semantic noise affecting the model.
3. **Tokenization**
   Split text into words using spaces as delimiters, breaking long text into basic word-level units.
4. **Filtering non-ASCII characters**
   Remove non-ASCII characters to reduce interference from special symbols and abnormal encodings during model training.
5. **Lemmatization**
   Apply WordNet Lemmatizer to reduce words to their base forms, lowering feature space dimensionality and enhancing semantic consistency.
6. **Low-information word filtering**
   Discard words shorter than three characters to reduce noise from non-informative tokens.

3.3.2 Text Feature Representation

During vectorization, two common and stable feature representations are used:

- **CountVectorizer**: Constructs Bag-of-Words features based on word frequency, serving as a baseline representation.
- **TF-IDF Vectorizer**: Introduces inverse document frequency on top of word frequency to reduce the impact of high-frequency but low-distinguishing words.

Both feature representations are compared under the same model structure to evaluate their impact on model performance.

Model Construction and Validation

4.1 Data Preparation and Evaluation Setup

This project models the task as a multi-label text classification problem, using a One-vs-Rest approach. A separate binary classification model is trained for each toxicity label, outputting prediction probabilities.

The dataset is split into training and validation sets, ensuring consistent label distribution across subsets. Model evaluation uses ROC AUC as the primary metric, and the average AUC across all labels is used as the overall performance measure. This setup accommodates label imbalance and emphasizes ranking ability, consistent with competition requirements.

4.2 Baseline Model Construction

To establish performance baselines, several traditional models known for stable performance in text classification are trained and compared:

- **Multinomial Naive Bayes**
  A classic text classification model, fast to train, suitable for quickly validating feature engineering effectiveness.
- **Logistic Regression**
  A linear model with stable results and strong interpretability, commonly used as an industrial benchmark for text classification.
- **Linear Support Vector Classifier (Linear SVC)**
  Demonstrates strong generalization on high-dimensional sparse text features and works well with TF-IDF representations.

All baseline models are trained using the same text feature representations and evaluated via cross-validation to measure average AUC, establishing an initial performance reference.

4.3 Model Selection and Optimization Strategy

On top of baseline models, boosting models are introduced to capture nonlinear relationships between features:

- AdaBoost
- GBDT
- XGBoost

Models are compared based on validation set AUC performance. The model with better overall performance and higher stability is selected as the candidate. Hyperparameter tuning focuses on maximizing AUC while avoiding overfitting to individual labels.

4.3.1 Ensemble Learning Model

To further improve generalization and prediction stability, a VotingClassifier ensemble is constructed based on multiple candidate models. By averaging or weighting predictions from several high-performing models, the ensemble effectively reduces single-model uncertainty.

Experimental results show that the ensemble model achieves more stable AUC performance across most labels and is used as the final submission model.

Model Performance Evaluation and Results Analysis

5.1 Experimental Design and Evaluation Metrics

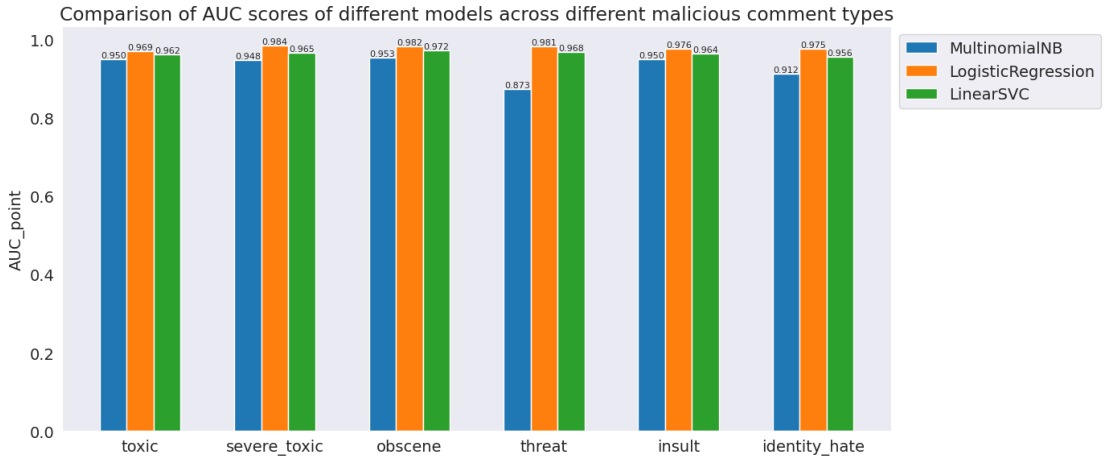
- **Task:** Multi-label text classification with 6 toxicity labels (toxic, severe_toxic, obscene, threat, insult, identity_hate)
- **Models:**
  - **Baseline models:** MultinomialNB, Logistic Regression, LinearSVC
  - **Ensemble models:** Voting Classifier (Logistic Regression + SVC + XGBoost), Boosting models (AdaBoost, GradientBoosting, XGBoost)
- **Evaluation metrics:** Recall, F1-score, ROC-AUC, Confusion Matrix

10-fold cross-validation is applied on the training set, while the test set is used to evaluate actual generalization performance. Confusion matrices are used to analyze the classification performance for “toxic / non-toxic” comments.

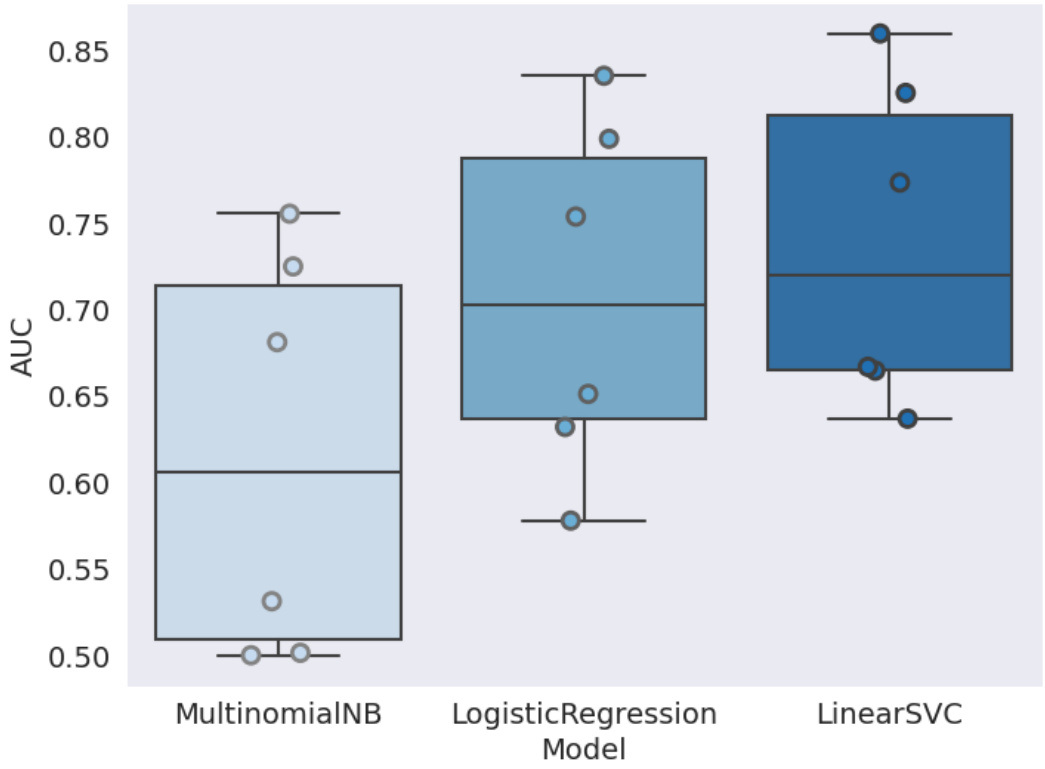
5.2 Model Performance Comparison

Cross-validation results (baseline models):

- **MultinomialNB:** High AUC for some labels but low Recall; minority classes are almost unrecognized.
- **Logistic Regression:** Overall balanced performance, outperforming Naive Bayes.
- **LinearSVC:** Highest Recall and F1-score for most labels, with stable AUC performance.



https://github.com/Morris-CP/Toxic-comments-categorized/tree/main

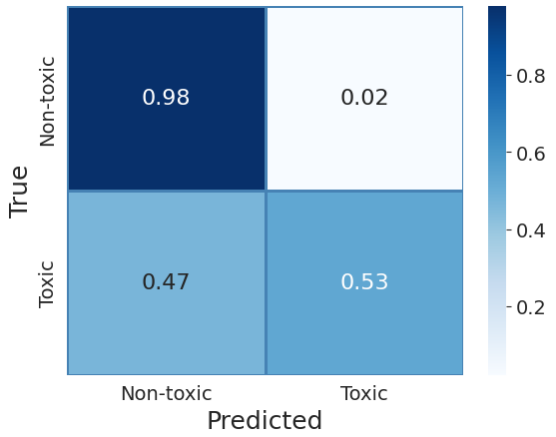


Confusion Matrix Analysis (toxic label)

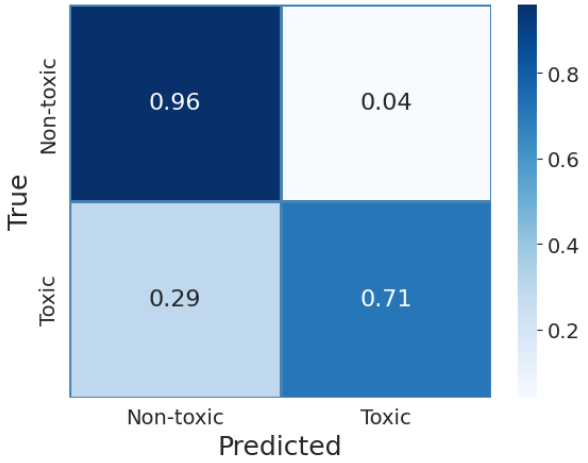
- **MultinomialNB:** Misses a large number of toxic comments.
- **Logistic Regression:** Improved recognition of toxic comments.
- **LinearSVC:** Effectively reduces the false negative rate.
- **VotingClassifier:** Recall slightly higher than Logistic Regression but less stable than LinearSVC.

https://github.com/Morris-CP/Toxic-comments-categorized/tree/main

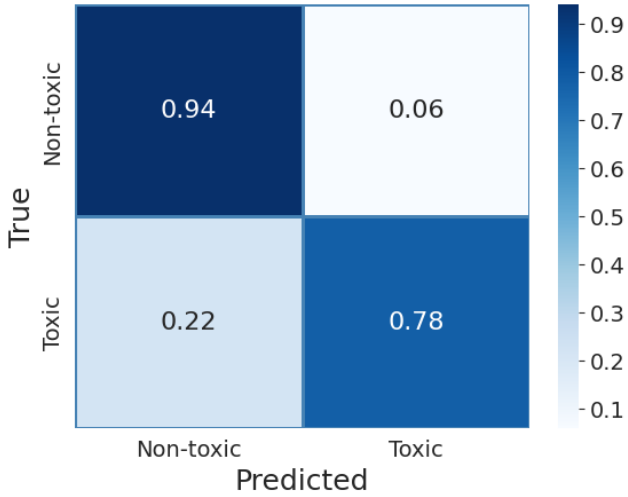
MultinomialNB



LogisticRegression



LinearSVC



https://github.com/Morris-CP/Toxic-comments-categorized/tree/main

Misclassification Analysis and Business Impact

- Misclassifications are mainly concentrated in comments with subtle semantics, sarcasm, or strong context dependency.
- **LinearSVC** achieves the best recall and overall stability, minimizing business risk.
- Ensemble models offer limited improvement but may slightly outperform baseline models on specific labels.

Test Set Results with Ensemble Models

- Baseline models average AUC:
- **LinearSVC 0.7378 > Logistic Regression 0.7081 > MultinomialNB 0.6157**
- Ensemble models average AUC:

	Model	AUC
0	AdaBoostClassifier	0.687482
1	GradientBoostingClassifier	0.665917
2	XGBClassifier	0.734515

- **VotingClassifier**: 0.7213
- **Boosting model** (XGBoost as representative): 0.7345

Analysis:

- Ensemble models show slight improvement on some labels but overall performance is similar to **LinearSVC**.
- For business-critical metrics (toxic comment recall), **LinearSVC** demonstrates more stable performance.
- Ensemble models have higher computational cost and increased deployment complexity.

5.3 Model Selection and Business Value

Final Model: LinearSVC

Reasons:

1. Highest average AUC, Recall, and F1-score.
2. Strong toxic comment detection, minimizing business risk.
3. Simple model structure, easy to deploy and maintain.

Business Value:
Automatically identifying toxic comments reduces manual moderation workload, enhances community safety, and supports rapid iteration and deployment.

Conclusions and Future Work

6.1 Key Conclusions

Through systematic evaluation of the multi-label toxic comment classification task, the following key conclusions were drawn:

1. **LinearSVC**
    - Demonstrates the most stable performance in Recall, F1-score, and AUC, especially in detecting toxic comments.
    - Boxplot and confusion matrix analyses show that LinearSVC effectively reduces the risk of missing toxic comments.
2. **Logistic Regression**
    - Balanced and stable performance.
    - Can serve as an alternative in scenarios requiring probability outputs and model interpretability.
3. **MultinomialNB**
    - Fast training, but low recall for minority labels.
    - Not suitable for critical business metrics, such as minimizing missed toxic comments.
4. **Ensemble Models (Voting / Boosting)**
    - Slight improvement on some labels.
    - Overall performance does not significantly exceed LinearSVC.
    - Higher deployment complexity, offering limited advantage in rapid iteration or production environments.

6.2 Business Value and Application Analysis

Reducing Missed Toxic Comments

- LinearSVC can automatically detect most toxic comments, reducing manual moderation workload.
- Enhances user experience and lowers public opinion risk on social platforms or community forums.

Interpretability and Deployment Feasibility

- Simple model structure, easy to integrate into existing systems.
- Suitable for rapid deployment with convenient monitoring and iteration.
- **Support for Multi-Scenario Applications**
- Can be combined with alert systems for automated content moderation.
- Model can be extended to different languages or platforms, enhancing cross-domain management capabilities.

6.3 Limitations and Improvement Directions

- **Data Imbalance**: Minority labels (e.g., threat, identity_hate) have few samples, resulting in suboptimal performance on boundary cases.
- **Feature Representation Limitations**: Bag-of-words models struggle to capture subtle semantics or context-dependent toxic language.
- **Ensemble Strategy Optimization**: Ensemble models provide limited improvement under current features; incorporating deep semantic models may further enhance performance.

6.4 Future Work

Incorporate Deep Semantic Models

Models like BERT / RoBERTa can capture context and subtle semantic cues, improving minority label detection.

Cost-Sensitive Learning and Independent Thresholds

Set different weights or thresholds for each label to enhance toxic comment recall.

Business System Deployment and Human-in-the-Loop

Build an automated review system complemented with manual verification.

Regularly update models to adapt to new patterns of toxic comments.